

Objetivo del producto

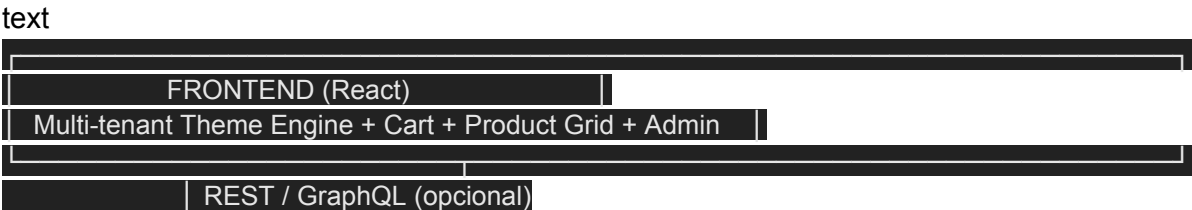
<https://grok.com/project/f9879b2c-0703-42fa-9d0e-cd68867b9a9d?tab=conversations>

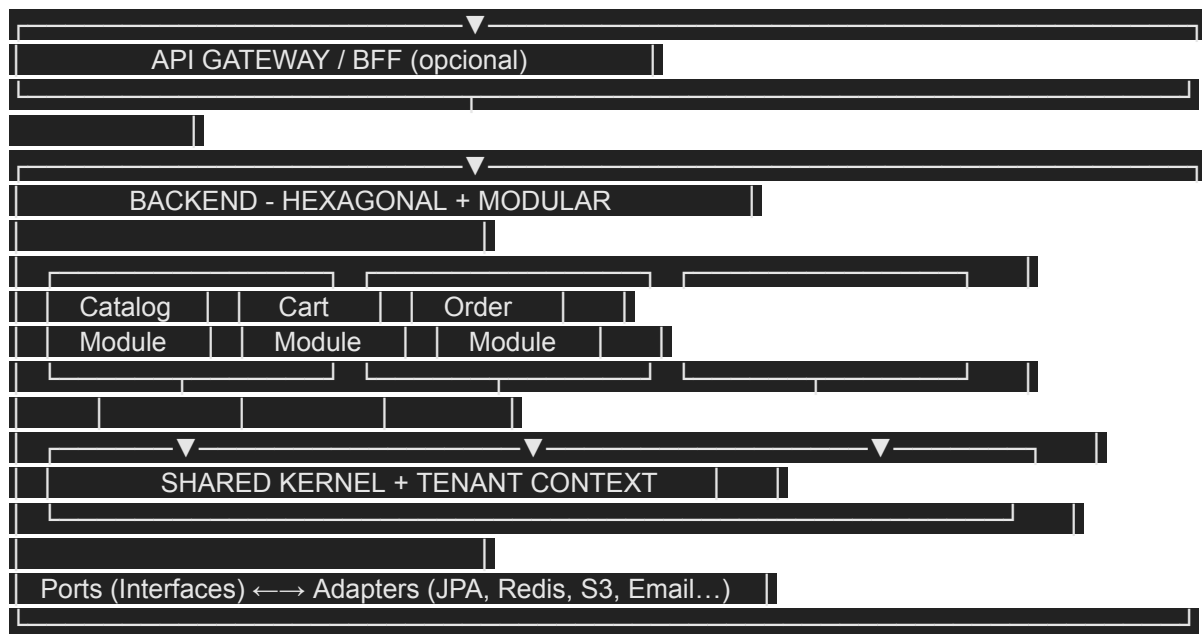
Una plataforma e-commerce **multi-tenant** (cada empresa tiene su propia tienda con logo, colores, dominio, productos, etc.), lista para vender como SaaS o white-label.

Stack tecnológico recomendado (óptimo para este caso)

Capa	Tecnología	Justificación
Frontend	React 18 + TypeScript + Vite + Tailwind + Shadcn/ui	Rápido, moderno, excelente DX, fácil de personalizar por tenant
Backend	Spring Boot 3 + Java 21	Maduro, fuerte en enterprise, excelente para hexagonal + DDD
Base de datos	PostgreSQL 16	Multi-tenant nativo, JSONB, excelente rendimiento
Cache	Redis	Sesiones, carrito, catálogo
Mensajería	RabbitMQ / Kafka (opcional)	Eventos de dominio
Auth	Keycloak o Spring Security + JWT	Multi-tenant seguro
Almacenamiento	MinIO / S3	Logos, imágenes de productos
Observabilidad	OpenTelemetry + ELK / Grafana Loki	Logs estructurados
Dashboard analítica	Metabase o Apache Superset + eventos	Análisis de ventas
Tests	JUnit 5 + Testcontainers + Playwright	Unit + Integration + E2E

Arquitectura general (Hexagonal + Modular + DDD + Loose Coupling)





Estructura de carpetas recomendada (Backend - Spring Boot)

text

```
src/main/java/com/tuempresa/ecommerce/
├── EcommerceApplication.java
├── shared/ # Shared Kernel
│   ├── domain/
│   │   ├── TenantId.java
│   │   ├── Money.java
│   │   └── DomainEvent.java
│   ├── application/
│   ├── infrastructure/
│   ├── persistence/
│   ├── security/
│   └── config/
├── tenant/ # Módulo de multi-tenancy
│   ├── domain/
│   ├── application/
│   └── infrastructure/
├── catalog/ # Productos + Categorías
│   ├── domain/
│   │   ├── Product.java
│   │   └── Category.java
│   ├── ProductRepository.java # Port
│   ├── application/
│   │   ├── ProductService.java
│   │   └── dto/
│   └── infrastructure/
```

```

|   |— persistence/      # Adapter JPA
|   |— web/              # Controllers
|
|   |— cart/
|   |— domain/
|   |— application/
|   |— infrastructure/
|
|   |— order/
|   |— payment/
|   |— inventory/
|   |— customer/
|   |— analytics/        # Eventos para dashboard
|   |— admin/

```

Principios aplicados

1. **Hexagonal (Ports & Adapters)**
 - El dominio **nunca** conoce JPA, Redis, S3, etc.
 - Interfaces (ports) en el dominio.
 - Implementaciones (adapters) en infrastructure.
2. **DDD (Domain-Driven Design)**
 - Agregados: Product, Cart, Order, Tenant.
 - Value Objects: Money, Sku, TenantId, Slug.
 - Domain Events: ProductCreated, OrderPlaced, CartAbandoned.
3. **Modular Monolith** (recomendado al inicio)
 - Cada módulo es un bounded context.
 - Comunicación entre módulos **solo** por eventos o interfaces públicas.
 - Fácil de extraer a microservicios más adelante.
4. **Multi-tenancy**
 - Estrategia recomendada: **Schema por tenant** o **Row-level** con tenant_id + RLS de PostgreSQL.
 - Para empezar: **Shared Database + tenant_id** + Row Level Security.
 - Cada request lleva el TenantContext (ThreadLocal o contexto reactivo).

Funcionalidades principales y páginas

Frontend (páginas públicas)

Página	Descripción
/	Home (banners, destacados, categorías)
/categoria/:slug	Listado por categoría (grid de tarjetas)

/buscar?q=	Búsqueda
/producto/:slug	Detalle de producto
/carrito	Carrito de compras
/checkout	Proceso de pago
/cuenta	Área de cliente
/pedidos	Historial de pedidos
/login / /registro	Auth

Admin (por tenant)

- Dashboard de ventas
- Gestión de productos (CRUD + imágenes)
- Categorías
- Pedidos
- Clientes
- Configuración de tienda (logo, colores, dominio, métodos de pago)
- Cupones / promociones

Características multi-tenant clave

- Cada empresa (tenant) puede subir su **logo** y definir colores (CSS variables dinámicas).
- Dominio personalizado o subdominio (empresa.tutienda.com).
- Catálogo, precios, stock y pedidos **aislados** por tenant.
- Tema personalizable en tiempo real (ThemeProvider en React).

Carrito de compras

- **Backend:** Carrito como agregado de dominio.
- Persistencia: Redis (rápido) + sincronización a PostgreSQL.
- Soporte de:
 - Cantidad
 - Variantes (talla, color...)
 - Cupones
 - Cálculo de impuestos y envío

Estructura de logging, testing y analytics

Logging

Java

```
// Structured logging con MDC
MDC.put("tenantId", tenantId);
MDC.put("userId", userId);
MDC.put("requestId", requestId);
log.info("Order placed", kv("orderId", orderId), kv("amount", amount));
```

Stack: Logback + JSON + ELK o Grafana Loki + OpenTelemetry.

Testing

- **Unit:** JUnit 5 + Mockito (dominio puro).
- **Integration:** Testcontainers (PostgreSQL + Redis).
- **Contract:** Spring Cloud Contract (opcional).
- **E2E:** Playwright (frontend) + RestAssured.

Analytics / Dashboard

- Eventos de dominio → Kafka/RabbitMQ → ClickHouse o PostgreSQL materializado.
- Dashboard con **Metabase** o **Superset** (open source).
- Métricas clave:
 - Ventas por día/mes
 - Productos más vendidos
 - Tasa de abandono de carrito
 - Conversión por tenant
 - Revenue por tenant

Modelo de datos simplificado (PostgreSQL)

SQL

```
-- Multi-tenant
tenants (id, name, subdomain, logo_url, theme_config jsonb, ...)
```



```
-- Catálogo
categories (id, tenant_id, name, slug, parent_id, ...)
products (id, tenant_id, name, slug, price, stock, images jsonb, ...)
```



```
-- Carrito
carts (id, tenant_id, customer_id, ...)
cart_items (id, cart_id, product_id, quantity, unit_price, ...)
```



```
-- Pedidos
orders (id, tenant_id, customer_id, status, total, ...)
order_items (...)
```

Roadmap de implementación recomendado

Fase 1 – MVP vendible (8-12 semanas)

1. Multi-tenancy + autenticación
2. Catálogo (productos + categorías + grid)
3. Carrito + checkout básico
4. Subida de logo y tema
5. Panel admin básico
6. Pagos (Stripe / MercadoPago)

Fase 2

- Inventario avanzado
- Cupones
- Envíos
- Analytics dashboard
- Notificaciones

Fase 3

- Marketplace (opcional)
- App móvil
- Extracción de módulos a microservicios

1. Diagrama de Arquitectura Detallado



Cart Module
Order Module
Payment Module
Analytics Module
Shared Kernel
TenantId, Money, DomainEvent
API Gateway / Spring Boot
Tenant Resolver Filter
Spring Security + JWT
Frontend - React + TypeScript
Theme Engine
Logo + Colors dinámicos
Storefront
Home / Categorías / Product Grid
Cart + Checkout
Admin Panel

2. Estructura completa de carpetas

Backend (Spring Boot 3 + Java 21)

text

```
ecommerce-platform/  
├── backend/  
│   ├── pom.xml  
│   ├── src/  
│   │   ├── main/  
│   │   │   ├── java/com/ecommerce/  
│   │   │   │   ├── EcommerceApplication.java  
│   │   │   │   └── shared/ # Shared Kernel  
│   │   │       ├── domain/  
│   │   │       │   ├── TenantId.java  
│   │   │       │   ├── Money.java  
│   │   │       │   ├── DomainEvent.java  
│   │   │       │   └── AggregateRoot.java  
│   │   │       ├── application/  
│   │   │       ├── infrastructure/  
│   │   │       └── config/
```

```

├── security/
├── persistence/
├── web/
├── tenant/ # Bounded Context: Tenant
├── domain/
├── Tenant.java
├── TenantRepository.java # Port
├── ThemeConfig.java
├── application/
├── TenantService.java
├── dto/
├── infrastructure/
├── persistence/
├── web/
├── catalog/ # Productos + Categorías
├── domain/
├── Product.java
├── Category.java
├── ProductRepository.java
├── CategoryRepository.java
├── application/
├── infrastructure/
├── cart/
├── domain/
├── Cart.java
├── CartItem.java
├── CartRepository.java
├── application/
├── infrastructure/
├── order/
├── domain/
├── Order.java
├── OrderItem.java
├── OrderStatus.java
├── OrderRepository.java
├── application/
├── infrastructure/
├── analytics/
├── resources/
├── application.yml
├── db/migration/ # Flyway
├── V1_init.sql
├── logback-spring.xml

```

```
| | |
| | | └─ test/
| | | └─ java/... (Testcontainers, etc.)
```

Frontend (React + TypeScript + Vite)

text

```
ecommerce-platform/
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   ├── tailwind.config.js
│   ├── src/
│   │   ├── main.tsx
│   │   ├── App.tsx
│   │   |
│   │   ├── theme/ # Theme Engine
│   │   │   ├── ThemeProvider.tsx
│   │   │   ├── useTheme.ts
│   │   │   └── types.ts
│   │   |
│   │   ├── features/
│   │   │   ├── catalog/
│   │   │   │   ├── components/
│   │   │   │   │   ├── ProductCard.tsx
│   │   │   │   │   ├── ProductGrid.tsx
│   │   │   │   │   └── CategoryList.tsx
│   │   │   │   └── pages/
│   │   │   │       ├── HomePage.tsx
│   │   │   │       ├── CategoryPage.tsx
│   │   │   │       └── ProductDetailPage.tsx
│   │   │   └── api/
│   │   |
│   │   ├── cart/
│   │   │   ├── components/
│   │   │   │   ├── CartDrawer.tsx
│   │   │   │   └── CartItem.tsx
│   │   │   └── pages/
│   │   │       └── CheckoutPage.tsx
│   │   └── store/ (Zustand o Redux)
│   |
│   ├── admin/
│   │   └── ...
│   |
│   ├── auth/
│   |
│   ├── shared/
│   └── components/
```

```

| | | | — hooks/
| | | | — api/ (axios + interceptors)
| | | | — types/
| | |
| | | — layouts/
| | | — StoreLayout.tsx
| | | — AdminLayout.tsx

```

3. Script de Base de Datos Inicial (PostgreSQL + Multi-tenant)

SQL

```
-- V1 init.sql
```

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
-----
-- TENANTS
-----
```

```
CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        VARCHAR(255) NOT NULL,
  subdomain   VARCHAR(100) UNIQUE NOT NULL,
  custom_domain VARCHAR(255),
  logo_url    TEXT,
  theme_config JSONB NOT NULL DEFAULT '{}',
  status       VARCHAR(50) NOT NULL DEFAULT 'ACTIVE',
  created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-----
-- CATEGORIES
-----
```

```
CREATE TABLE categories (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  parent_id   UUID REFERENCES categories(id),
  name        VARCHAR(255) NOT NULL,
  slug        VARCHAR(255) NOT NULL,
  description  TEXT,
  image_url   TEXT,
  position    INT DEFAULT 0,
  is_active   BOOLEAN DEFAULT TRUE,

```

```

        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        UNIQUE(tenant_id, slug)
    );

```

```

-----
-- PRODUCTS
-----

```

```

CREATE TABLE products (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    category_id UUID REFERENCES categories(id),
    name VARCHAR(255) NOT NULL,
    slug VARCHAR(255) NOT NULL,
    description TEXT,
    price NUMERIC(12,2) NOT NULL,
    compare_at_price NUMERIC(12,2),
    cost_price NUMERIC(12,2),
    sku VARCHAR(100),
    barcode VARCHAR(100),
    stock INT NOT NULL DEFAULT 0,
    track_inventory BOOLEAN DEFAULT TRUE,
    images JSONB DEFAULT '[]',
    attributes JSONB DEFAULT '{}',
    is_active BOOLEAN DEFAULT TRUE,
    is_featured BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(tenant_id, slug)
);

```

```

CREATE INDEX idx_products_tenant ON products(tenant_id);
CREATE INDEX idx_products_category ON products(category_id);

```

```

-----
-- CARTS
-----

```

```

CREATE TABLE carts (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id UUID NOT NULL REFERENCES tenants(id),
    customer_id UUID, -- null = guest
    session_id VARCHAR(255),
    currency VARCHAR(3) DEFAULT 'MXN',
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE cart_items (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

```

```

    cart_id      UUID NOT NULL REFERENCES carts(id) ON DELETE CASCADE,
    product_id   UUID NOT NULL REFERENCES products(id),
    quantity     INT NOT NULL CHECK (quantity > 0),
    unit_price   NUMERIC(12,2) NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-----
-- ORDERS
-----

```

```

CREATE TYPE order_status AS ENUM (
    'PENDING', 'PAID', 'PROCESSING', 'SHIPPED', 'DELIVERED', 'CANCELLED', 'REFUNDED'
);

```

```

CREATE TABLE orders (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id     UUID NOT NULL REFERENCES tenants(id),
    customer_id   UUID,
    order_number  VARCHAR(50) NOT NULL,
    status        order_status NOT NULL DEFAULT 'PENDING',
    currency      VARCHAR(3) DEFAULT 'MXN',
    subtotal      NUMERIC(12,2) NOT NULL,
    discount      NUMERIC(12,2) DEFAULT 0,
    shipping_cost NUMERIC(12,2) DEFAULT 0,
    tax           NUMERIC(12,2) DEFAULT 0,
    total         NUMERIC(12,2) NOT NULL,
    shipping_address JSONB,
    billing_address JSONB,
    notes         TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(tenant_id, order_number)
);

```

```

CREATE TABLE order_items (
    id            UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id      UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id    UUID NOT NULL,
    product_name  VARCHAR(255) NOT NULL,
    sku           VARCHAR(100),
    quantity      INT NOT NULL,
    unit_price    NUMERIC(12,2) NOT NULL,
    total_price   NUMERIC(12,2) NOT NULL
);

```

```

-----
-- Row Level Security (opcional pero recomendado)
-----

```

```

-- ALTER TABLE products ENABLE ROW LEVEL SECURITY;

```

```
-- CREATE POLICY tenant_isolation ON products
-- USING (tenant_id = current_setting('app.current_tenant')::uuid);
```

4. Código base de los Agregados principales (DDD puro)

TenantId.java (Shared Kernel)

Java

```
package com.ecommerce.shared.domain;
```

```
import java.util.Objects;
```

```
import java.util.UUID;
```

```
public record TenantId(UUID value) {
```

```
    public TenantId {
```

```
        Objects.requireNonNull(value, "TenantId cannot be null");
```

```
    }
```

```
    public static TenantId of(UUID value) {
```

```
        return new TenantId(value);
```

```
    }
```

```
    public static TenantId generate() {
```

```
        return new TenantId(UUID.randomUUID());
```

```
    }
```

```
    @Override
```

```
    public String toString() {
```

```
        return value.toString();
```

```
    }
```

```
}
```

Money.java

Java

```
package com.ecommerce.shared.domain;
```

```
import java.math.BigDecimal;
```

```
import java.util.Currency;
```

```
import java.util.Objects;
```

```
public record Money(BigDecimal amount, Currency currency) {
```

```
    public Money {
```

```

    Objects.requireNonNull(amount);
    Objects.requireNonNull(currency);
    if (amount.scale() > 2) {
        throw new IllegalArgumentException("Money scale cannot be greater than 2");
    }
}

public static Money of(BigDecimal amount, String currencyCode) {
    return new Money(amount, Currency.getInstance(currencyCode));
}

public static Money zero(String currencyCode) {
    return of(BigDecimal.ZERO, currencyCode);
}

public Money add(Money other) {
    assertSameCurrency(other);
    return new Money(this.amount.add(other.amount), this.currency);
}

public Money multiply(int quantity) {
    return new Money(this.amount.multiply(BigDecimal.valueOf(quantity)), this.currency);
}

private void assertSameCurrency(Money other) {
    if (!this.currency.equals(other.currency)) {
        throw new IllegalArgumentException("Currency mismatch");
    }
}
}

```

Tenant.java (Aggregate Root)

Java

```

package com.ecommerce.tenant.domain;

import com.ecommerce.shared.domain.AggregateRoot;
import com.ecommerce.shared.domain.TenantId;

import java.time.Instant;
import java.util.Objects;

public class Tenant extends AggregateRoot {

    private final TenantId id;
    private String name;
    private String subdomain;
    private String customDomain;
    private String logoUrl;
}

```

```

    private ThemeConfig themeConfig;
    private TenantStatus status;
    private Instant createdAt;
    private Instant updatedAt;

    private Tenant(TenantId id, String name, String subdomain) {
        this.id = Objects.requireNonNull(id);
        this.name = Objects.requireNonNull(name);
        this.subdomain = Objects.requireNonNull(subdomain);
        this.themeConfig = ThemeConfig.defaultTheme();
        this.status = TenantStatus.ACTIVE;
        this.createdAt = Instant.now();
        this.updatedAt = Instant.now();
    }

    public static Tenant create(String name, String subdomain) {
        return new Tenant(TenantId.generate(), name, subdomain);
    }

    public void updateTheme(ThemeConfig newTheme) {
        this.themeConfig = Objects.requireNonNull(newTheme);
        this.updatedAt = Instant.now();
        // registerEvent(new TenantThemeUpdated(...));
    }

    public void updateLogo(String logoUrl) {
        this.logoUrl = logoUrl;
        this.updatedAt = Instant.now();
    }

    // Getters
    public TenantId getId() { return id; }
    public String getName() { return name; }
    public String getSubdomain() { return subdomain; }
    public String getLogoUrl() { return logoUrl; }
    public ThemeConfig getThemeConfig() { return themeConfig; }
    public TenantStatus getStatus() { return status; }
}

```

ThemeConfig.java

Java

```
package com.ecommerce.tenant.domain;
```

```
import java.util.Map;
```

```

public record ThemeConfig(
    String primaryColor,
    String secondaryColor,

```

```

        String accentColor,
        String backgroundColor,
        String textColor,
        String fontFamily,
        Map<String, String> extra
    ){
        public static ThemeConfig defaultTheme(){
            return new ThemeConfig(
                "#2563eb", // blue-600
                "#1e40af",
                "#f59e0b",
                "#ffffff",
                "#111827",
                "Inter, system-ui, sans-serif",
                Map.of()
            );
        }
    }
}

```

Product.java (Aggregate Root)

Java

```

package com.ecommerce.catalog.domain;

import com.ecommerce.shared.domain.AggregateRoot;
import com.ecommerce.shared.domain.Money;
import com.ecommerce.shared.domain.TenantId;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;
import java.util.UUID;

public class Product extends AggregateRoot {

    private final UUID id;
    private final TenantId tenantId;
    private String name;
    private String slug;
    private String description;
    private Money price;
    private Money compareAtPrice;
    private int stock;
    private boolean trackInventory;
    private List<String> images;
    private boolean active;
    private boolean featured;

    private Product(UUID id, TenantId tenantId, String name, String slug, Money price) {

```

```

        this.id = id;
        this.tenantId = Objects.requireNonNull(tenantId);
        this.name = Objects.requireNonNull(name);
        this.slug = Objects.requireNonNull(slug);
        this.price = Objects.requireNonNull(price);
        this.images = new ArrayList<>();
        this.active = true;
        this.trackInventory = true;
        this.stock = 0;
    }

    public static Product create(TenantId tenantId, String name, String slug, Money price) {
        return new Product(UUID.randomUUID(), tenantId, name, slug, price);
    }

    public void updatePrice(Money newPrice) {
        this.price = Objects.requireNonNull(newPrice);
    }

    public void addStock(int quantity) {
        if (quantity < 0) throw new IllegalArgumentException("Quantity must be positive");
        this.stock += quantity;
    }

    public void reduceStock(int quantity) {
        if (!trackInventory) return;
        if (this.stock < quantity) {
            throw new InsufficientStockException(this.id, quantity, this.stock);
        }
        this.stock -= quantity;
    }

    public boolean isAvailable() {
        return active && (!trackInventory || stock > 0);
    }

    // Getters...
    public UUID getId() { return id; }
    public TenantId getTenantId() { return tenantId; }
    public String getName() { return name; }
    public Money getPrice() { return price; }
    public int getStock() { return stock; }
}

```

Cart.java (Aggregate Root)

Java

```
package com.ecommerce.cart.domain;
```

```

import com.ecommerce.shared.domain.AggregateRoot;
import com.ecommerce.shared.domain.Money;
import com.ecommerce.shared.domain.TenantId;

import java.util.*;

public class Cart extends AggregateRoot {

    private final UUID id;
    private final TenantId tenantId;
    private UUID customerId; // null = guest
    private String sessionId;
    private final List<CartItem> items = new ArrayList<>();
    private String currency;

    private Cart(UUID id, TenantId tenantId, String currency) {
        this.id = id;
        this.tenantId = tenantId;
        this.currency = currency;
    }

    public static Cart create(TenantId tenantId, String currency) {
        return new Cart(UUID.randomUUID(), tenantId, currency);
    }

    public void addItem(UUID productId, String productName, Money unitPrice, int quantity) {
        Optional<CartItem> existing = items.stream()
            .filter(i -> i.getProductId().equals(productId))
            .findFirst();

        if (existing.isPresent()) {
            existing.get().increaseQuantity(quantity);
        } else {
            items.add(new CartItem(productId, productName, unitPrice, quantity));
        }
    }

    public void removeItem(UUID productId) {
        items.removeIf(i -> i.getProductId().equals(productId));
    }

    public void changeQuantity(UUID productId, int newQuantity) {
        items.stream()
            .filter(i -> i.getProductId().equals(productId))
            .findFirst()
            .ifPresent(i -> i.changeQuantity(newQuantity));
    }

    public Money total() {

```

```

        return items.stream()
            .map(CartItem::lineTotal)
            .reduce(Money.zero(currency), Money::add);
    }

    public List<CartItem> getItems() {
        return Collections.unmodifiableList(items);
    }

    public UUID getId() { return id; }
    public TenantId getTenantId() { return tenantId; }
}

```

CartItem.java (Entity)

Java

```

package com.ecommerce.cart.domain;

import com.ecommerce.shared.domain.Money;

import java.util.UUID;

public class CartItem {
    private final UUID productId;
    private final String productName;
    private final Money unitPrice;
    private int quantity;

    public CartItem(UUID productId, String productName, Money unitPrice, int quantity) {
        this.productId = productId;
        this.productName = productName;
        this.unitPrice = unitPrice;
        this.quantity = quantity;
    }

    public void increaseQuantity(int amount) {
        this.quantity += amount;
    }

    public void changeQuantity(int newQuantity) {
        if (newQuantity <= 0) throw new IllegalArgumentException("Quantity must be > 0");
        this.quantity = newQuantity;
    }

    public Money lineTotal() {
        return unitPrice.multiply(quantity);
    }

    // Getters

```

```

    public UUID getProductId() { return productId; }
    public String getProductName() { return productName; }
    public Money getUnitPrice() { return unitPrice; }
    public int getQuantity() { return quantity; }
}

```

Order.java (Aggregate Root) – versión resumida

Java

```

package com.ecommerce.order.domain;

import com.ecommerce.shared.domain.AggregateRoot;
import com.ecommerce.shared.domain.Money;
import com.ecommerce.shared.domain.TenantId;

import java.util.ArrayList;
import java.util.List;
import java.util.UUID;

public class Order extends AggregateRoot {

    private final UUID id;
    private final TenantId tenantId;
    private final String orderNumber;
    private OrderStatus status;
    private final List<OrderItem> items = new ArrayList<>();
    private Money subtotal;
    private Money total;
    private final String currency;

    private Order(UUID id, TenantId tenantId, String orderNumber, String currency) {
        this.id = id;
        this.tenantId = tenantId;
        this.orderNumber = orderNumber;
        this.currency = currency;
        this.status = OrderStatus.PENDING;
    }

    public static Order createFromCart(TenantId tenantId, String orderNumber, String currency,
List<OrderItem> items) {
        Order order = new Order(UUID.randomUUID(), tenantId, orderNumber, currency);
        order.items.addAll(items);
        order.recalculateTotals();
        return order;
    }

    public void markAsPaid() {
        if (status != OrderStatus.PENDING) {
            throw new IllegalStateException("Only PENDING orders can be paid");
        }
    }
}

```

```

    }
    this.status = OrderStatus.PAID;
    // registerEvent(new OrderPaidEvent(...));
}

private void recalculateTotals() {
    this.subtotal = items.stream()
        .map(OrderItem::getTotalPrice)
        .reduce(Money.zero(currency), Money::add);
    this.total = this.subtotal; // + shipping + tax - discount
}

// Getters...
}

```

5. Configuración Multi-tenant en Spring

TenantContext.java

Java

```

package com.ecommerce.shared.infrastructure;

import com.ecommerce.shared.domain.TenantId;

public final class TenantContext {
    private static final ThreadLocal<TenantId> CURRENT = new ThreadLocal<>();

    public static void set(TenantId tenantId) {
        CURRENT.set(tenantId);
    }

    public static TenantId get() {
        return CURRENT.get();
    }

    public static void clear() {
        CURRENT.remove();
    }
}

```

TenantFilter.java (Servlet Filter)

Java

```

package com.ecommerce.shared.infrastructure.web;

```

```

import com.ecommerce.shared.domain.TenantId;
import com.ecommerce.shared.infrastructure.TenantContext;
import com.ecommerce.tenant.domain.TenantRepository;
import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.UUID;

@Component
@Order(1)
public class TenantFilter implements Filter {

    private final TenantRepository tenantRepository;

    public TenantFilter(TenantRepository tenantRepository) {
        this.tenantRepository = tenantRepository;
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest httpRequest = (HttpServletRequest) request;
        String subdomain = extractSubdomain(httpRequest);
        String headerTenant = httpRequest.getHeader("X-Tenant-ID");

        try {
            TenantId tenantId = resolveTenant(subdomain, headerTenant);
            TenantContext.set(tenantId);
            chain.doFilter(request, response);
        } finally {
            TenantContext.clear();
        }
    }

    private TenantId resolveTenant(String subdomain, String headerTenant) {
        if (headerTenant != null) {
            return TenantId.of(UUID.fromString(headerTenant));
        }
        // Buscar por subdomain
        return tenantRepository.findBySubdomain(subdomain)
            .map(t -> t.getId())
            .orElseThrow(() -> new TenantNotFoundException(subdomain));
    }

    private String extractSubdomain(HttpServletRequest request) {

```

```

    String host = request.getServerName(); // ej: acme.mitienda.com
    if (host.contains(".")) {
        return host.split("\\.")[0];
    }
    return "default";
}
}
}

```

application.yml (extracto)

YAML

```

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/ecommerce
    username: ecommerce
    password: secret

  jpa:
    hibernate:
      ddl-auto: validate
    properties:
      hibernate:
        # Para multi-tenant con discriminator (alternativa a schema)
        multiTenancy: DISCRIMINATOR
        tenant_identifier_resolver:
com.ecommerce.shared.infrastructure.persistence.CurrentTenantIdentifierResolver

app:
  multi-tenant:
    strategy: SHARED_DATABASE # o SCHEMA_PER_TENANT

```

6. Theme Engine en React

src/theme/types.ts

TypeScript

```

export interface ThemeConfig {
  primaryColor: string;
  secondaryColor: string;
  accentColor: string;
  backgroundColor: string;
  textColor: string;
  fontFamily: string;
  logoUrl?: string;
  extra?: Record<string, string>;
}

```

```
}
```

```
export const defaultTheme: ThemeConfig = {  
  primaryColor: "#2563eb",  
  secondaryColor: "#1e40af",  
  accentColor: "#f59e0b",  
  backgroundColor: "#ffffff",  
  textColor: "#111827",  
  fontFamily: "Inter, system-ui, sans-serif",  
};
```

src/theme/ThemeProvider.tsx

tsx

```
import React, { createContext, useContext, useEffect, useState } from "react";  
import { ThemeConfig, defaultTheme } from "../types";
```

```
interface ThemeContextValue {  
  theme: ThemeConfig;  
  setTheme: (theme: ThemeConfig) => void;  
}
```

```
const ThemeContext = createContext<ThemeContextValue | undefined>(undefined);
```

```
export const ThemeProvider: React.FC<{ children: React.ReactNode; initialTheme?: ThemeConfig }> = ({  
  children,  
  initialTheme = defaultTheme,  
}) => {  
  const [theme, setTheme] = useState<ThemeConfig>(initialTheme);
```

```
  useEffect(() => {  
    // Inyectar CSS variables en el :root  
    const root = document.documentElement;  
    root.style.setProperty("--color-primary", theme.primaryColor);  
    root.style.setProperty("--color-secondary", theme.secondaryColor);  
    root.style.setProperty("--color-accent", theme.accentColor);  
    root.style.setProperty("--color-bg", theme.backgroundColor);  
    root.style.setProperty("--color-text", theme.textColor);  
    root.style.setProperty("--font-family", theme.fontFamily);  
  }, [theme]);
```

```
  return (  
    <ThemeContext.Provider value={{ theme, setTheme }}>  
      {children}  
    </ThemeContext.Provider>  
  );  
};
```

```
export const useTheme = () => {
```

```
const ctx = useContext(ThemeContext);
if (!ctx) throw new Error("useTheme must be used within ThemeProvider");
return ctx;
};
```

Uso en main.tsx / App.tsx

tsx

```
import { ThemeProvider } from "../theme/ThemeProvider";
import { useEffect, useState } from "react";
import { fetchTenantTheme } from "../shared/api/tenant";

function App() {
  const [theme, setTheme] = useState(null);

  useEffect(() => {
    // Detectar subdomain y cargar tema del tenant
    const subdomain = window.location.hostname.split(".")[0];
    fetchTenantTheme(subdomain).then(setTheme);
  }, []);

  if (!theme) return <div>Loading store...</div>;

  return (
    <ThemeProvider initialTheme={theme}>
      { /* Rutas de la tienda */ }
    </ThemeProvider>
  );
}
```

Ejemplo de ProductCard usando el tema

tsx

```
import { useTheme } from "../../theme/useTheme";

export function ProductCard({ product }) {
  const { theme } = useTheme();

  return (
    <div className="rounded-xl border shadow-sm overflow-hidden hover:shadow-md transition">
      <img src={product.images[0]} alt={product.name} className="w-full h-48 object-cover" />
      <div className="p-4">
        <h3 className="font-semibold text-[var(--color-text)]">{product.name}</h3>
        <p className="text-lg font-bold mt-1" style={{ color: "var(--color-primary)" }}>
          ${product.price}
        </p>
        <button
          className="mt-3 w-full py-2 rounded-lg text-white font-medium"
          style={{ backgroundColor: "var(--color-primary)" }}
        >
```

```
>  
    Agregar al carrito  
</button>  
</div>  
</div>  
);  
}
```

Arquitectura Final – E-commerce SaaS Multi-Tenant

Principios aplicados

- **Hexagonal Architecture** (Ports & Adapters)
 - **Domain-Driven Design** (Bounded Contexts)
 - **Modular Monolith** (fácil de evolucionar a microservicios)
 - **Loose Coupling** mediante Domain Events
 - **Multi-Tenant** con Shared Schema + tenant_id + Row Level Security (RLS)
-

1. Diagrama de Arquitectura Final

Backend - Spring Boot 3 + Java 21

Modular Monolith + Hexagonal

Cliente

Observability

Structured Logs + tenant_id

Micrometer

Tremor / Metabase

Infrastructure

PostgreSQL 16

Shared Schema + RLS

Redis - Cart & Cache

MinIO / S3 - Logos & Images

Domain Events

Spring Events → RabbitMQ

Bounded Contexts

Identity & Tenant
Catalog
Cart & Checkout
Order Fulfillment
Analytics
Frontend - React + Vite + TypeScript
Theme Engine
CSS Variables + Logo
Storefront
Home / Categorías / Product Grid / Cart
Admin Panel
Dashboard / Products / Orders / Branding
Browser / Mobile
Tenant Resolver + Security Filter
Shared Kernel
TenantId • Money • DomainEvent

2. Estructura de Carpetas Definitiva

Backend

text

```
ecommerce-saas/  
├── backend/  
│   ├── pom.xml  
│   ├── src/main/java/com/ecommerce/  
│   │   ├── EcommerceApplication.java  
│   │   ├──  
│   │   ├── shared/ # Shared Kernel  
│   │   │   ├── domain/  
│   │   │   │   ├── TenantId.java  
│   │   │   │   ├── Money.java  
│   │   │   │   ├── AggregateRoot.java  
│   │   │   │   ├── DomainEvent.java  
│   │   │   ├── application/  
│   │   │   ├── infrastructure/  
│   │   │   ├── config/  
│   │   │   ├── security/  
│   │   │   ├── multitenancy/ # TenantContext + Filter + RLS  
│   │   │   └── persistence/
```

```
| | | | | web/
| | | |
| | | | | tenant/ # Bounded Context: Identity & Tenant
| | | | | | domain/
| | | | | | | Tenant.java
| | | | | | | ThemeConfig.java
| | | | | | | TenantStatus.java
| | | | | | | TenantRepository.java # Port
| | | | | | application/
| | | | | | | GetTenantConfigUseCase.java
| | | | | | | UpdateThemeUseCase.java
| | | | | | | dto/
| | | | | | infrastructure/
| | | | | | persistence/
| | | | | | web/ # /api/v1/tenant/config
| | | | |
| | | | | catalog/ # Bounded Context: Catalog
| | | | | | domain/
| | | | | | | Product.java
| | | | | | | Category.java
| | | | | | | ProductRepository.java
| | | | | | | CategoryRepository.java
| | | | | | application/
| | | | | | | CreateProductUseCase.java
| | | | | | | ListProductsUseCase.java
| | | | | | | dto/
| | | | | | infrastructure/
| | | | |
| | | | | cart/ # Bounded Context: Cart & Checkout
| | | | | | domain/
| | | | | | | Cart.java
| | | | | | | CartItem.java
| | | | | | | CartRepository.java
| | | | | | application/
| | | | | | infrastructure/
| | | | |
| | | | | order/ # Bounded Context: Order Fulfillment
| | | | | | domain/
| | | | | | | Order.java
| | | | | | | OrderItem.java
| | | | | | | OrderStatus.java
| | | | | | | OrderRepository.java
| | | | | | application/
| | | | | | infrastructure/
| | | | |
| | | | | analytics/ # Bounded Context: Analytics
| | | |
| | | | | src/main/resources/
| | | | | application.yml
```

```
| | | | | db/migration/
| | | | |   | V1 init schema.sql
| | | | |   | logback-spring.xml
```

Frontend

text

```
ecommerce-saas/
| | | | | frontend/
| | | | |   | package.json
| | | | |   | vite.config.ts
| | | | |   | tailwind.config.js
| | | | |   | src/
| | | | |     | main.tsx
| | | | |     | App.tsx
| | | | |
| | | | |   | theme/ # Theme Engine
| | | | |     | ThemeProvider.tsx
| | | | |     | useTheme.ts
| | | | |     | types.ts
| | | | |
| | | | |   | features/
| | | | |     | catalog/
| | | | |       | components/
| | | | |         | ProductCard.tsx
| | | | |         | ProductGrid.tsx
| | | | |         | CategoryNav.tsx
| | | | |       | pages/
| | | | |         | HomePage.tsx
| | | | |         | CategoryPage.tsx
| | | | |         | ProductDetailPage.tsx
| | | | |       | api/
| | | | |
| | | | |   | cart/
| | | | |     | components/
| | | | |     | pages/
| | | | |       | CartPage.tsx
| | | | |       | CheckoutPage.tsx
| | | | |     | store/ # Zustand
| | | | |
| | | | |   | admin/
| | | | |     | pages/
| | | | |       | DashboardPage.tsx
| | | | |       | ProductsPage.tsx
| | | | |       | OrdersPage.tsx
| | | | |       | BrandingPage.tsx
| | | | |     | components/
| | | | |
| | | | |   | auth/
```

```

| | |
| | | — shared/
| | | — api/ # Axios + Tenant Interceptor
| | | — components/
| | | — hooks/
| | | — types/
| | |
| | | — layouts/
| | | — StoreLayout.tsx # Header con logo dinámico
| | | — AdminLayout.tsx

```

3. Script de Base de Datos Final (PostgreSQL + RLS)

SQL

```
-- V1 init schema.sql
```

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

```
-----
-- TENANTS
-----
```

```
CREATE TABLE tenants (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  name        VARCHAR(255) NOT NULL,
  subdomain   VARCHAR(100) UNIQUE NOT NULL,
  custom_domain VARCHAR(255),
  logo_url    TEXT,
  theme_config JSONB NOT NULL DEFAULT '{
    "primaryColor": "#2563eb",
    "secondaryColor": "#1e40af",
    "accentColor": "#f59e0b",
    "backgroundColor": "#ffffff",
    "textColor": "#111827",
    "fontFamily": "Inter, system-ui, sans-serif"
  }',
  status      VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-----
-- CATEGORIES
-----
```

```
CREATE TABLE categories (
  id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  tenant_id   UUID NOT NULL REFERENCES tenants(id),
  parent_id   UUID REFERENCES categories(id),

```

```

    name      VARCHAR(255) NOT NULL,
    slug       VARCHAR(255) NOT NULL,
    description TEXT,
    image_url  TEXT,
    position   INT DEFAULT 0,
    is_active  BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (tenant_id, slug)
);

```

```

CREATE INDEX idx_categories_tenant ON categories(tenant_id);

```

```

-----
-- PRODUCTS
-----

```

```

CREATE TABLE products (
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id   UUID NOT NULL REFERENCES tenants(id),
    category_id UUID REFERENCES categories(id),
    name        VARCHAR(255) NOT NULL,
    slug        VARCHAR(255) NOT NULL,
    description  TEXT,
    price       NUMERIC(12,2) NOT NULL,
    compare_at_price NUMERIC(12,2),
    sku         VARCHAR(100),
    stock       INT NOT NULL DEFAULT 0,
    track_inventory BOOLEAN DEFAULT TRUE,
    images      JSONB DEFAULT '[]',
    attributes  JSONB DEFAULT '{}',
    is_active   BOOLEAN DEFAULT TRUE,
    is_featured BOOLEAN DEFAULT FALSE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (tenant_id, slug)
);

```

```

CREATE INDEX idx_products_tenant ON products(tenant_id);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_products_featured ON products(tenant_id, is_featured) WHERE is_featured =
TRUE;

```

```

-----
-- CARTS
-----

```

```

CREATE TABLE carts (
    id          UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id   UUID NOT NULL REFERENCES tenants(id),
    customer_id UUID,

```

```

    session_id    VARCHAR(255),
    currency       VARCHAR(3) DEFAULT 'MXN',
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

CREATE TABLE cart_items (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    cart_id        UUID NOT NULL REFERENCES carts(id) ON DELETE CASCADE,
    product_id     UUID NOT NULL REFERENCES products(id),
    quantity       INT NOT NULL CHECK (quantity > 0),
    unit_price     NUMERIC(12,2) NOT NULL,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

```

-----
-- ORDERS
-----

```

```

CREATE TYPE order_status AS ENUM (
    'PENDING', 'PAID', 'PROCESSING', 'SHIPPED', 'DELIVERED', 'CANCELLED', 'REFUNDED'
);

```

```

CREATE TABLE orders (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    tenant_id      UUID NOT NULL REFERENCES tenants(id),
    customer_id    UUID,
    order_number    VARCHAR(50) NOT NULL,
    status         order_status NOT NULL DEFAULT 'PENDING',
    currency       VARCHAR(3) DEFAULT 'MXN',
    subtotal       NUMERIC(12,2) NOT NULL,
    discount       NUMERIC(12,2) DEFAULT 0,
    shipping_cost  NUMERIC(12,2) DEFAULT 0,
    tax            NUMERIC(12,2) DEFAULT 0,
    total          NUMERIC(12,2) NOT NULL,
    shipping_address JSONB,
    billing_address JSONB,
    notes          TEXT,
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (tenant_id, order_number)
);

```

```

CREATE TABLE order_items (
    id             UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    order_id       UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id     UUID NOT NULL,
    product_name   VARCHAR(255) NOT NULL,
    sku            VARCHAR(100),
    quantity       INT NOT NULL,

```

```

    unit_price NUMERIC(12,2) NOT NULL,
    total_price NUMERIC(12,2) NOT NULL
);

-----
-- ROW LEVEL SECURITY
-----

ALTER TABLE categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE carts ENABLE ROW LEVEL SECURITY;
ALTER TABLE cart_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;

-- Política de aislamiento por tenant
CREATE POLICY tenant_isolation_categories ON categories
    USING (tenant_id = current_setting('app.current_tenant', true)::uuid);

CREATE POLICY tenant_isolation_products ON products
    USING (tenant_id = current_setting('app.current_tenant', true)::uuid);

CREATE POLICY tenant_isolation_carts ON carts
    USING (tenant_id = current_setting('app.current_tenant', true)::uuid);

CREATE POLICY tenant_isolation_orders ON orders
    USING (tenant_id = current_setting('app.current_tenant', true)::uuid);

-- Nota: cart_items y order_items se protegen a través de la relación con carts/orders

```

4. Multi-Tenant en Spring (Configuración Final)

TenantContext

Java

```

package com.ecommerce.shared.infrastructure.multitenancy;

import com.ecommerce.shared.domain.TenantId;

public final class TenantContext {
    private static final ThreadLocal<TenantId> CURRENT = new ThreadLocal<>();

    public static void set(TenantId tenantId) {
        CURRENT.set(tenantId);
    }

    public static TenantId get() {

```

```

        TenantId id = CURRENT.get();
        if (id == null) {
            throw new IllegalStateException("No tenant set in context");
        }
        return id;
    }

    public static void clear() {
        CURRENT.remove();
    }
}

```

TenantFilter (resuelve subdominio o header)

Java

```

package com.ecommerce.shared.infrastructure.multitenancy;

import com.ecommerce.shared.domain.TenantId;
import com.ecommerce.tenant.domain.TenantRepository;
import jakarta.servlet.*;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.core.annotation.Order;
import org.springframework.stereotype.Component;

import java.io.IOException;
import java.util.UUID;

@Component
@Order(1)
public class TenantFilter implements Filter {

    private final TenantRepository tenantRepository;

    public TenantFilter(TenantRepository tenantRepository) {
        this.tenantRepository = tenantRepository;
    }

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        try {
            TenantId tenantId = resolveTenant(req);
            TenantContext.set(tenantId);

            // Para RLS de PostgreSQL
            // Se setea en la conexión vía Hibernate Interceptor o StatementInspector

```

```

        chain.doFilter(request, response);
    } finally {
        TenantContext.clear();
    }
}

private TenantId resolveTenant(HttpServletRequest request) {
    String header = request.getHeader("X-Tenant-ID");
    if (header != null && !header.isBlank()) {
        return TenantId.of(UUID.fromString(header));
    }

    String host = request.getServerName(); // empresa1.plataforma.com
    String subdomain = host.contains(".") ? host.split("\\.")[0] : "default";

    return tenantRepository.findBySubdomain(subdomain)
        .map(t -> t.getId())
        .orElseThrow(() -> new TenantNotFoundException(subdomain));
}
}

```

5. Theme Engine en React (Final)

src/theme/types.ts

TypeScript

```

export interface ThemeConfig {
    primaryColor: string;
    secondaryColor: string;
    accentColor: string;
    backgroundColor: string;
    textColor: string;
    fontFamily: string;
    logoUrl?: string;
}

export const defaultTheme: ThemeConfig = {
    primaryColor: "#2563eb",
    secondaryColor: "#1e40af",
    accentColor: "#f59e0b",
    backgroundColor: "#ffffff",
    textColor: "#111827",
    fontFamily: "Inter, system-ui, sans-serif",
};

```

src/theme/ThemeProvider.tsx

tsx

```
import React, { createContext, useContext, useEffect, useState } from "react";
import { ThemeConfig, defaultTheme } from "./types";

interface ThemeContextType {
  theme: ThemeConfig;
  setTheme: (theme: ThemeConfig) => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

export const ThemeProvider: React.FC<{
  children: React.ReactNode;
  initialTheme?: ThemeConfig;
}> = ({ children, initialTheme = defaultTheme }) => {
  const [theme, setTheme] = useState<ThemeConfig>(initialTheme);

  useEffect(() => {
    const root = document.documentElement;
    root.style.setProperty("--color-primary", theme.primaryColor);
    root.style.setProperty("--color-secondary", theme.secondaryColor);
    root.style.setProperty("--color-accent", theme.accentColor);
    root.style.setProperty("--color-bg", theme.backgroundColor);
    root.style.setProperty("--color-text", theme.textColor);
    root.style.setProperty("--font-family", theme.fontFamily);
  }, [theme]);

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};

export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) throw new Error("useTheme must be used within ThemeProvider");
  return context;
};
```

6. Páginas Principales (Frontend)

Storefront

Ruta

Descripción

/	Home (hero + destacados + categorías)
/c/:categorySlug	Grid de productos por categoría
/p/:productSlug	Detalle de producto
/cart	Carrito
/checkout	Checkout
/account	Área de cliente
/search	Búsqueda

Admin Panel

Ruta	Descripción
/admin/dashboard	KPIs + gráficas
/admin/products	CRUD productos
/admin/orders	Gestión de órdenes
/admin/branding	Logo + colores + tipografía
/admin/settings	Configuración general

7. Stack Tecnológico Final

Capa	Tecnología
Frontend	React 18 + TypeScript + Vite + Tailwind + Zustand + React Router
Backend	Java 21 + Spring Boot 3.3+
Base de datos	PostgreSQL 16 (Shared Schema + RLS)
Cache / Carrito	Redis
Archivos	MinIO / AWS S3
Auth	Spring Security + JWT

Testing	JUnit 5 + Testcontainers + Playwright
Logs	Logback JSON + tenant_id + trace_id
Analytics	Domain Events + tablas agregadas + Tremor / Metabase